

MediSync — How It Actually Works

A no-jargon walkthrough. If you've never touched this codebase, start here.

Part 1: The Big Picture (in plain English)

Think of MediSync as the **post office** between three people who currently don't talk to each other well:

- **Doctor** — writes the prescription
- **Hospital** — bills the patient + sends the claim
- **Insurer** — decides what to pay
- **Patient** — the one stuck in the middle

Without MediSync, every step is manual + paper-based. With MediSync, each step is structured, automatic, and explained in plain language.

```
PATIENT uploads their policy (PDF)
|
v
DOCTOR records conversation → AI turns speech into clinical note
|
v
HOSPITAL checks if prescribed meds are covered by PATIENT's policy (using RAG)
|                                     ^
|                                     | this is the "smart" bit
|                                     | ChromaDB + LLM
v
HOSPITAL submits claim with final bill → INSURER
|
v
INSURER auto-adjudicates (Python rules, not AI) → approves / rejects / flags
|
v
PATIENT sees status in plain English, can upload messy bill → AI explains it
```

Part 2: The Happy Path (step by step, with code)

Let's follow **one claim** from start to finish.

Step 0 — Someone logs in

Any user (doctor, hospital, insurer, patient) hits the login page.

What happens in code:

1. User types email + password → React form submits → `client/src/context/AuthContext.jsx` `login()` function fires.
2. That calls `POST /api/auth/login` — handled in `server/routes/auth.py:54`
3. Backend looks up user in MongoDB, checks password hash using `bcrypt`, then returns a **JWT token** (`server/middleware/auth_middleware.py:31` → `create_access_token`)
4. Frontend stores that token in `localStorage`
5. Every future API call auto-attaches the token in the header (`client/src/utils/api.js:14-20`)
6. Based on the user's `role`, they get redirected to their dashboard — logic in `client/src/App.jsx` (`RootRedirect` + `ProtectedRoute`)

The sidebar changes per role — see `client/src/components/Layout.jsx:12-40`. One component, 4 different menus.

Step 1 — Patient uploads their insurance policy PDF

This is where the RAG magic starts.

What the user does:

- Patient logs in → goes to "My Insurance Policy" → uploads a PDF of their health policy.

What the code does:

- Frontend: `PatientUploadPolicy.jsx` sends the file as `multipart/form-data` to `POST /api/patient/upload-policy`
- Backend: `server/routes/patient.py:277` → `upload_my_policy()` saves the PDF to disk, then calls the RAG service.

Now RAG kicks in — see `server/services/rag_service.py:52` → `index_policy_pdf()`:

1. **PyPDF2 reads the PDF** — extracts every word of text (`rag_service.py:30` → `extract_text_from_pdf`)
2. **Chunk the text** — breaks the huge policy doc into overlapping 500-word pieces (`rag_service.py:41` → `chunk_text`). Why chunks? LLMs can't read the whole PDF at once — we feed it only the relevant pieces later.

3. **Store in ChromaDB** ([rag_service.py:72-76](#)) — ChromaDB automatically turns each chunk into a **vector** (a list of numbers that represent the meaning). This lets us later ask "which chunks of this policy talk about diabetes medication?" and get the right answer back.

What ChromaDB is doing (in noob terms):

*Imagine the policy PDF has 50 pages. ChromaDB reads it, breaks it into paragraphs, and builds a "smart index" that lets you search by **meaning**, not by keyword. If you ask "is Paracetamol covered?", ChromaDB finds the paragraph that talks about "fever medication", even if the word "paracetamol" doesn't appear there.*

Step 2 — Doctor records the consultation (Smart Scribe)

What the user does:

- Doctor opens consultation page → picks a patient → clicks  "Start Recording" → talks to patient → clicks "Stop"
- The browser's built-in Web Speech API turns voice into text (zero cost, zero API calls)
- Doctor reviews the transcript, clicks "Process with AI"

What the code does:

- Frontend: [client/src/pages/doctor/Consultation.jsx](#) sends `POST /api/doctor/structure-note` with the transcript + patient_id
- Backend: [server/routes/doctor.py:27](#) → `structure_transcript()` :

Here's what happens next — this is the AI flow:

1. **LLM call** — [server/services/llm_service.py:238](#) → `structure_clinical_note(transcript)` sends the transcript to Gemini (or HuggingFace, or a mock) with a prompt: "Turn this doctor-patient conversation into JSON with symptoms, diagnosis, prescriptions, ICD codes."
2. **LLM returns structured JSON** — the mess of words becomes clean data:

```
json { "symptoms": ["fever", "cough"], "diagnosis": "Acute Upper Respiratory Infection", "prescriptions": [{"medication": "Paracetamol 500mg", ...}], "icd_codes": ["J06.9"]} }
```
3. **RAG coverage check** — [server/routes/doctor.py:49-72](#). For EACH prescription, the code calls `check_medication_coverage()` ([rag_service.py:138](#)):
4. Asks ChromaDB: "Find me chunks of this policy that talk about `Paracetamol` "
5. ChromaDB returns the 3 most relevant paragraphs from the PDF
6. Sends those paragraphs + the drug name to the LLM: "Based on this policy text, is Paracetamol covered? If not, suggest a generic alternative."

7. LLM returns `{is_covered: true/false, alternative: "..."}`

8. **FHIR JSON built** — `server/routes/doctor.py:249` → `_build_fhir_encounter()`. FHIR is a global healthcare data standard. We output the clinical note in FHIR format so any other hospital system can read it.

9. **Saved to MongoDB** in the `clinical_notes` collection.

Doctor now sees on screen:

- Symptoms as tags
 - Diagnosis
 - Prescriptions table — with 🚫 red warnings if any drug isn't covered, like "⚠️ *Sucralfate is NOT covered. Switch to Pantoprazole 40mg for claim approval.*"
-

Step 3 — Hospital reviews the note + checks coverage

What the user does:

- Hospital admin logs in → goes to "Patient Records" → expands a patient → clicks "Check Coverage"
- Reviews each medication — some green ✓, some red ✗ with suggested alternatives
- Clicks "Switch" on red ones to accept the alternative
- Clicks "Submit Claim to Insurer" (if patient has a policy) OR "Bill Patient Directly" (if no policy)

What the code does:

- Frontend: `client/src/pages/hospital/PatientRecords.jsx:37-50` → `checkCoverage()` calls `POST /api/hospital/check-medication-coverage`
- Backend: `server/routes/hospital.py:202` — same RAG pipeline as doctor, but shows results in a slicker UI with substitution buttons.

When the hospital hits "Submit Claim":

- Frontend navigates to SubmitClaim page with the accepted substitutions in route state
 - That page builds the final claim doc → `POST /api/hospital/submit-claim`
 - Backend `server/routes/hospital.py:118` → `submit_claim()` saves a `claim` document in MongoDB with `status: "pending"`
-

Step 4 — Insurer auto-adjudicates

Key point: This step has **NO AI**. It's pure Python rules. Insurance decisions need to be explainable + auditable — LLMs are too unpredictable for this.

What the user does:

- Insurer logs in → sees pending claims → clicks "Auto-Adjudicate All" OR reviews individually

What the code does:

- Frontend: `POST /api/insurer/adjudicate-all-pending`
- Backend: `server/routes/insurer.py:204` loops through pending claims, calls `adjudicate_claim()` on each.

The real logic is in `server/services/adjudication_service.py:81` → `adjudicate_claim()`. It runs 6 rules:

Rule	What it checks	Line
1	Is claim amount under the policy limit?	:94-97
2	Is room rent under the daily cap?	:100-108
3	Are any items in the "excluded" list (cosmetic, dental, etc.)?	:111-116
4	Are any ICD codes "complex" (cancer, cardiac → needs human)?	:119-127
5	Does the diagnosis mention surgery/chemo (needs pre-auth)?	:130-134
6	Final decision: <code>approved</code> , <code>flagged</code> , or <code>rejected</code>	:136-153

Output saved back to the claim doc — patient can now see "approved" or "flagged for review" on their dashboard.

The rules live in a JSON-like dictionary at `adjudication_service.py:11-66`. Change a room rent cap? Just edit the number. No retraining, no deployment.

Step 5 — Patient sees everything in plain English

The patient dashboard pulls three flows together:

A) Claim tracker

- `GET /api/patient/claims` → `server/routes/patient.py:186`
- Every claim has a `status_explanation` field that translates adjudication jargon into human sentences — see `patient.py:260` `_explain_claim_status()`. Example: *"Your claim for ₹12,185 needs additional review because room rent exceeds your policy cap."*

B) Bill decoder (upload messy hospital bill image)

- Patient uploads bill JPEG → `POST /api/patient/upload-bill` → `patient.py:25`
- **OCR runs** — `server/services/ocr_service.py:19` `extract_bill_items()`. Uses `pytesseract` (the free Google OCR engine) to turn image → text.

- Regex parses line items: "Room Charges (3 days) ₹15,000" → `{description, amount, category}`
- **LLM explains each charge** — `llm_service.py:416 analyze_bill()` . Each line gets tagged `covered: true/false` with a plain-English reason.

C) Discharge summary translator

- Patient pastes the discharge summary (in English, medical jargon)
- `POST /api/patient/translate-discharge` → `patient.py:80`
- Three things happen in sequence:
- **LLM simplifies** medical jargon → plain English (`llm_service.py:452 simplify_discharge_summary`)
- **deep-translator** translates plain English → Hindi/Bengali/Tamil (`translation_service.py:30`)
- **gTTS** turns that translated text into an MP3 audio file (`translation_service.py:49 text_to_audio`) — perfect for elderly / illiterate patients.
- Patient gets: plain English text + vernacular text + a downloadable audio clip

D) Daily health check

- Patient fills a short form (wound, fever, pain, appetite, mobility)
- `POST /api/patient/health-check` → `patient.py:122`
- `patient.py:218 _evaluate_health_check()` runs rules — any concerning combo (bleeding + fever + pain ≥ 7) → `is_flagged = true`
- Flagged check auto-notifies the doctor (shows in their "Patient Alerts" page)

Part 3: Deep Dive — What the Hell Is RAG?

RAG = Retrieval-Augmented Generation. The LLM + a searchable knowledge base.

The problem RAG solves

LLMs like Gemini don't know your patient's specific insurance policy. You can't just paste 50 pages of legal text into every question — it's too much, too slow, too expensive.

The RAG recipe

1. **Once** — break the PDF into chunks, store them in ChromaDB with vector embeddings (`rag_service.py:52`)
2. **Every question** — do this:

3. Convert the question ("is Paracetamol covered?") into a vector
4. ChromaDB finds the 3 chunks whose vectors are closest (= most similar in meaning) ([rag_service.py:87](#))
5. Feed just those 3 chunks + the question into the LLM
6. LLM answers based ONLY on that context — no hallucination

Why ChromaDB specifically?

- **Free** — runs locally, no cloud
- **Automatic embeddings** — you pass text, it handles the vectorization itself using a built-in model (sentence-transformers)
- **Persistent** — survives server restarts (we save to `./chroma_data/`)
- **Simple API** — `.upsert()` to add, `.query()` to search. Three lines of code.

The ChromaDB lifecycle in our app

When	What happens	Code
Server boot	ChromaDB client initializes, reads from disk	rag_service.py:16-19
Patient uploads PDF	New collection <code>policy_<id></code> created, chunks indexed	rag_service.py:52
Doctor/hospital checks a drug	Query that collection for relevant chunks	rag_service.py:87
Server restarts, ChromaDB loses data	Auto re-index from saved PDF on disk	rag_service.py:121 <code>_try_reindex</code>

Part 4: Quick Reference — Who Calls What

Happy path function chain (doctor consultation)

```
Frontend click "Process with AI"
  |
  v
Consultation.jsx → api.post("/doctor/structure-note")
  |
  v
auth_middleware.py:get_current_user      (JWT decode, look up user)
  |
  v
doctor.py:structure_transcript            (the main handler)
  |
  ├──> llm_service.py:structure_clinical_note  (LLM → JSON)
  |
  └──> rag_service.py:check_medication_coverage [per prescription]
        |
        ├──> rag_service.py:query_policy      (ChromaDB search)
        |
        └──> llm_service.py:check_policy_coverage (LLM interprets)
  |
  v
doctor.py:_build_fhir_encounter           (wrap in FHIR standard)
  |
  v
db.clinical_notes.insert_one              (save to MongoDB)
```

Claim adjudication chain

```
InsurerDashboard.jsx → "Auto-Adjudicate All"
  |
  v
insurer.py:adjudicate_all_pending        (loops over pending claims)
  |
  v
adjudication_service.py:adjudicate_claim (runs 6 rules)
  ├── Rule 1: amount limit
  ├── Rule 2: room rent cap
  ├── Rule 3: excluded items
  ├── Rule 4: complex ICD codes
  ├── Rule 5: pre-auth check
  └── Rule 6: final status
  |
  v
db.claims.update_one                     (save status, flags, approved amount)
```

Part 5: Tech Stack — What Each Thing Does

Piece	Purpose	Where it lives
React + Vite	The UI you click on	<code>client/</code>
React Router	URL routing + role-based redirects	<code>App.jsx</code>
Axios	Talk to the backend API	<code>utils/api.js</code>
AuthContext	Remembers who's logged in	<code>context/AuthContext.jsx</code>
FastAPI	The Python backend server	<code>server/main.py</code>
MongoDB Atlas	Stores users, claims, notes, bills	<code>server/database.py</code>
Motor	Async MongoDB driver (for FastAPI)	—
JWT + bcrypt	Login tokens + password hashing	<code>auth_middleware.py</code>
ChromaDB	Vector database for policy PDFs	<code>rag_service.py</code>
PyPDF2	Reads text from PDF files	<code>rag_service.py:30</code>
pytesseract	OCR — reads text from images	<code>ocr_service.py</code>
Gemini / HuggingFace	The AI that understands + generates text	<code>llm_service.py</code>
deep-translator	Free Google Translate wrapper	<code>translation_service.py:30</code>
gTTS	Free Google TTS → MP3 audio	<code>translation_service.py:49</code>
Web Speech API	Browser's built-in speech-to-text (zero cost)	in <code>Consultation.jsx</code>
Recharts	Insurer analytics charts	in <code>Analytics.jsx</code>

Part 6: Mental Model Cheatsheet

LLM = the "brain". Understands language, generates language. We use Gemini (free tier). Unpredictable — never trust it for money math or legal decisions.

RAG = "let the brain read the right page first." Instead of giving the LLM the whole book, we give it the 3 relevant paragraphs.

ChromaDB = "a librarian that files paragraphs by meaning." Stores text chunks + their vector representations. You ask a question → it returns the most similar paragraphs.

Vector = a list of numbers (like 384 floats). Two sentences with similar meaning have similar vectors. Math on vectors = searching by meaning.

FHIR = the global standard for representing medical data in JSON. Any healthcare system can read FHIR, so our clinical notes are portable.

OCR = "read the letters in an image." Pytesseract opens a JPEG of a bill and types out what it sees.

TPA (in insurance) = Third Party Administrator. The company that processes claims for an insurer. Acts like the insurer for the doctor/patient.

ICD-10 = global medical code list. Every diagnosis has a 3-6 character code. "J06.9" means Acute Upper Respiratory Infection. Insurers use these codes to decide coverage.

Appendix: Exporting This to PDF

You can turn this file into a PDF with:

```
# Using pandoc (brew install pandoc)
pandoc FLOW.md -o FLOW.pdf --pdf-engine=xelatex

# Or open in VS Code + use the "Markdown PDF" extension

# Or paste into Google Docs and File → Download → PDF
```